– Manual de Instruções – Sistemas de Baixo Custo Para Sensoriamento, Registro e Automação das Estufas Agrícolas. (CTAN)

Sumário

1. Introdução..... 1
2. Materiais Necessários.....2
3. Esquema Elétrico.....2
4. Montagem..... 3
5. Testes e Funcionamento.....8
5.1 Sensor de Temperatura DS18B20..... 8
5.2 Sensor de Temperatura e Umidade DHT22..... 11
5.3 Sensor de Umidade do Solo Capacitivo..... 12
5.4 Sensor de Umidade do Solo Resistivo.....13
5.5 Sensor de Luminosidade BH1750..... 15
5.6 ESP8266..... 15
5.7 Código completo..... 19
5.8 Resultado esperado.....34
6. Execução do coletor de dados e Dashboard dos sensores.....35
6.1 Gerar senha de aplicativo do Gmail.....36
6.2 Executando o instalador do script.....37
6.3 Executando Dashboard.....40
7. Expansões e Melhorias.....43
8. Referências.....43

1. Introdução

Projeto voltado para monitorar a capacidade de campo e condições de ambiente. A capacidade de campo refere-se à quantidade máxima de água que o solo pode reter. O controle da quantidade de água é crucial para o crescimento

saudável das plantas e uso eficiente da água. As condições ambientais são monitoradas por sensores eletrônicos que medem a luminosidade, temperatura e umidade relativa do ar.

2. Materiais Necessários

- Arduino MEGA 2560 (1x).
- Protoboard (1x).
- Sensor de Umidade: DHT22 (2x).
- Sensor de Temperatura: DS18B20 (3x).
- Módulo Wi-fi/Bluetooth: ESP8266 (1x).
- Sensor de Umidade do Solo: Capacitive Soil (10x).
- Sensor de Umidade do Solo: Resistivo (3x).
- Sensor de Luminosidade: BH1750 (1x).
- Tubo Termo Retrátil.
- Jumpers.
- Resistor 10kΩ (4x).
- Fonte 9v (1x)
- Cabo de Rede (5x).

3. Esquema Elétrico

COMPONENTE	**PORTA**	**OBSERVAÇÕES**
DS18B20	12	Digital
DHT22	7 e 5	Digital
BH1750	20 e 21	Serial
Sensor Capacitivo	A0 e A1, e A5 ao A12	Analógico
Sensor Resistivo	A2, A3 e A4	Analógico
ESP8266	18 e 19	Serial

4. Montagem

1. Com o Arduíno Mega e um protoboard conecte o ESP8266 nas portas indicadas no **Tópico 3** da seguinte forma:

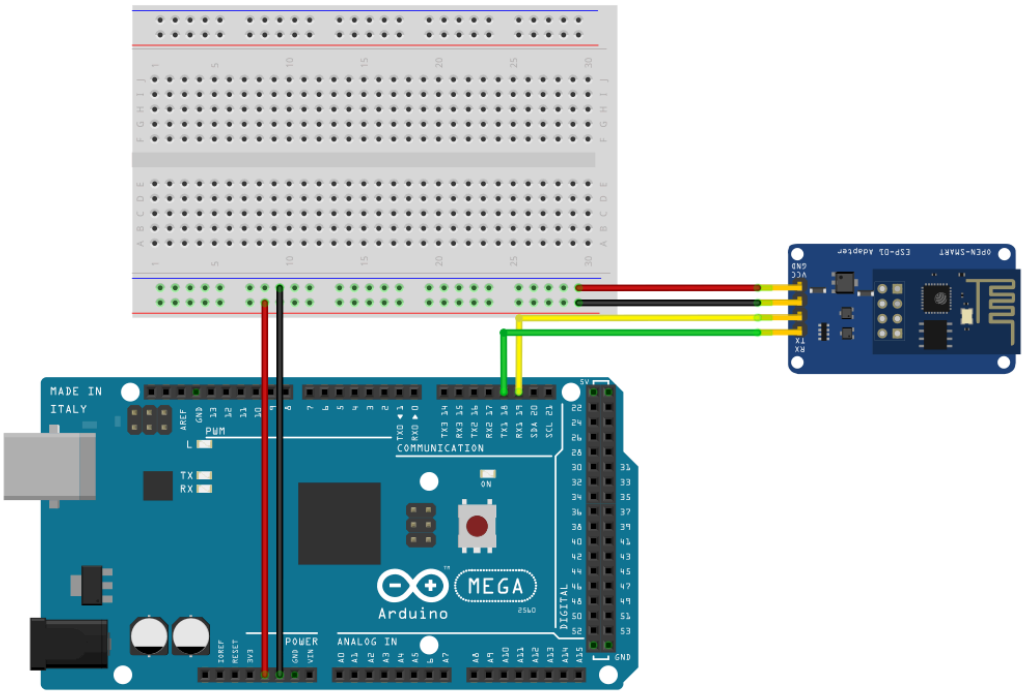


Figura 4.1

2. Para os sensores de temperatura DS18B20 usando um resistor de 10kΩ e apenas uma porta digital indicada no **Tópico 3**, conecte quantos sensores

quiser como na imagem:

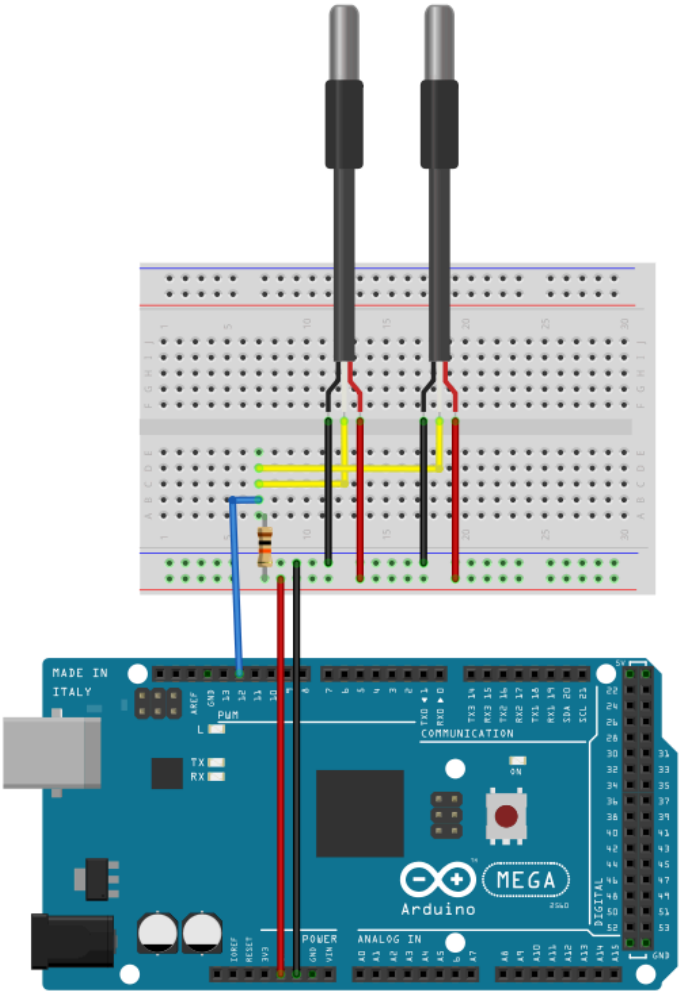


Figura 4.2

3. Para o sensor de temperatura e umidade DHT22, usando as portas indicadas no **Tópico 3**, conecte-o da seguinte forma:

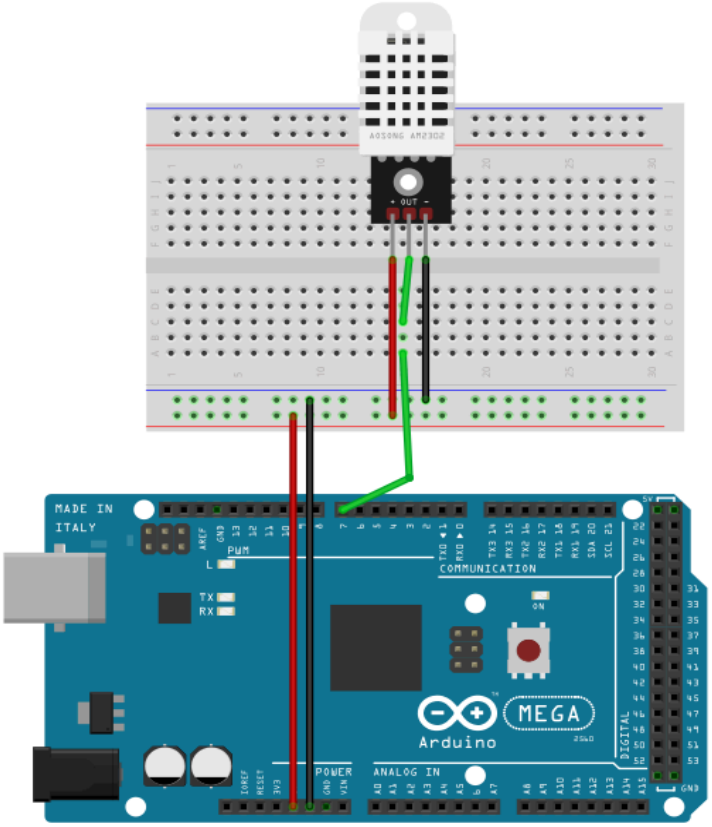


Figura 4.3

4. Para o sensor de luminosidade BH1750, seguindo as portas indicadas no **Tópico 3**, monte-o da seguinte forma:

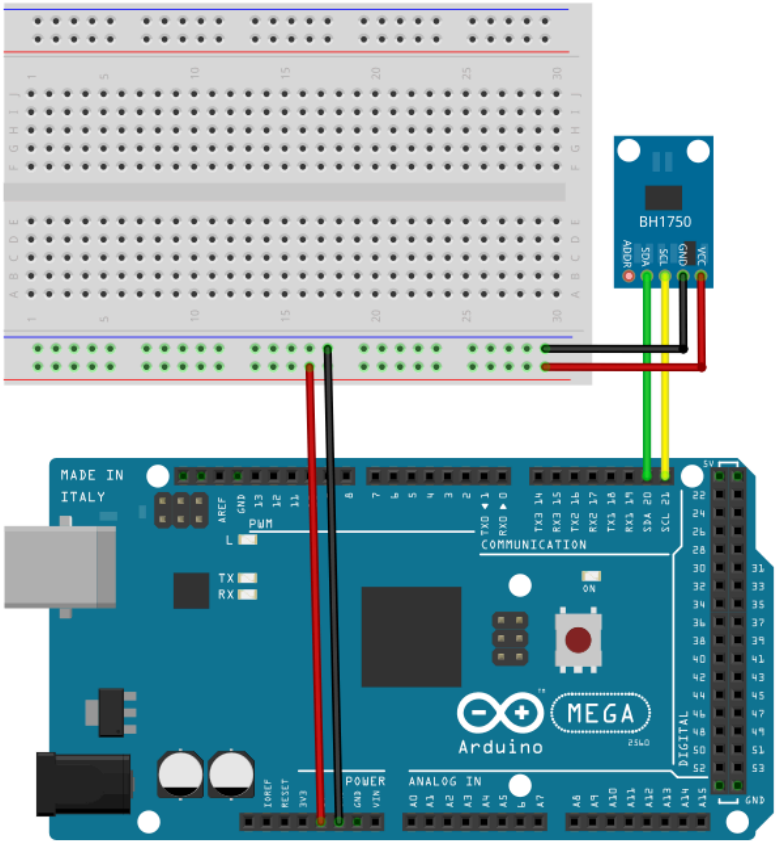


Figura 4.4

5. Para os sensores de umidade do solo capacitivo, seguindo as portas indicadas no **Tópico 3**, conecte quantos forem necessários da seguinte forma:

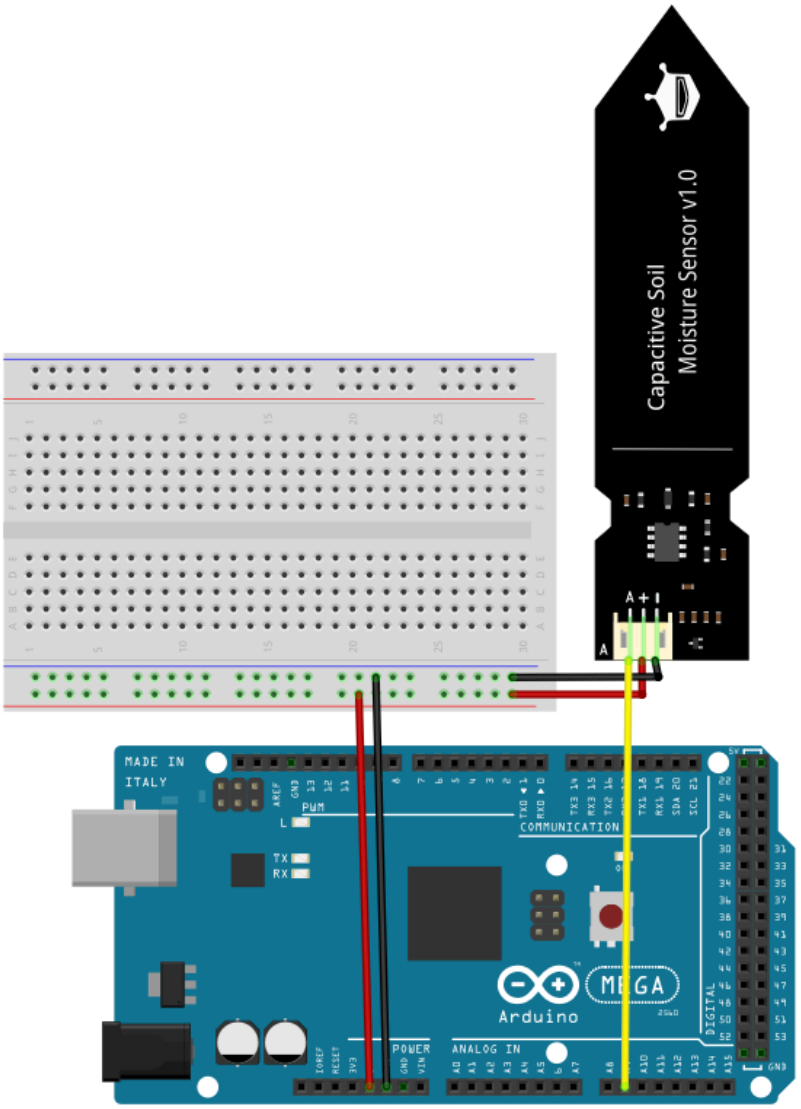


Figura 4.5

6. Para os sensores de umidade do solo resistivo, seguindo as portas indicadas no **Tópico 3**, usa-se um resistor de 10kΩ e conecte-os da seguinte forma:

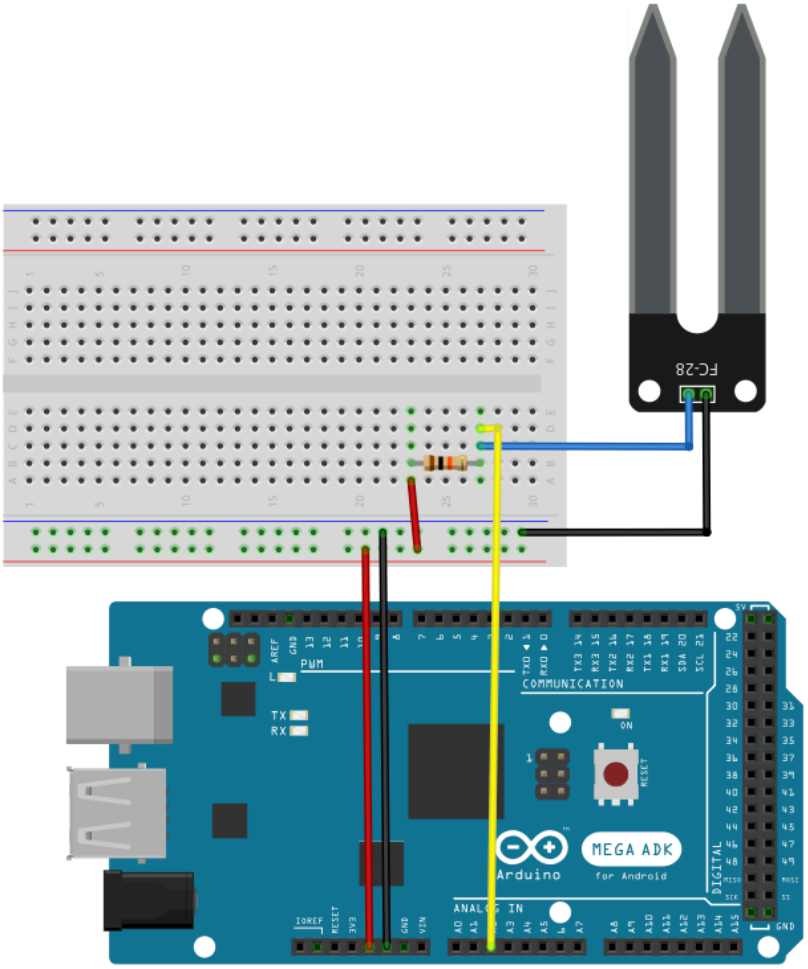


Figura 4.6

5. Testes e Funcionamento

Nessa seção será tratado toda a estrutura do código e o funcionamento dos sensores, aqui será mostrado o passo a passo para o código de cada sensor a fim de concluir com o código completo e o funcionamento do sistema. A cada tópico/código os sensores devem ser testados e podem ser modificados para calibragem caso necessário.

5.1 Sensor de Temperatura DS18B20

Para o código do sensor de temperatura baixe as bibliotecas **OneWire** e **DallasTemperature** e o implemente da seguinte forma:

```
#include <OneWire.h>
#include <DallasTemperature.h>

#define MAX_SENSOR 5 // Número máximo de sensores

// Configura o barramento OneWire no pino 12
OneWire oneWire(12);
// Cria o objeto "sensor" que gerencia os dispositivos conectados
DallasTemperature sensor(&oneWire);
// Array para armazenar os endereços dos sensores encontrados
DeviceAddress enderecos[MAX_SENSOR];
// DeviceAddress enderecos[MAX_SENSOR] = {
//    {0x28, 0x98, 0x0C, 0x46, 0xD4, 0x89, 0x5A, 0x79},
//    {0x28, 0xBB, 0xB9, 0x46, 0xD4, 0xB3, 0x52, 0x03},
//    {0x28, 0x8A, 0xEE, 0x46, 0xD4, 0xF4, 0x21, 0xF2},
//    {0x28, 0x48, 0xFC, 0x46, 0xD4, 0xA0, 0x68, 0x6A},
//    {0x28, 0xEC, 0xB5, 0x46, 0xD4, 0x86, 0x3E, 0x9D},
//    {0x28, 0x3D, 0x9E, 0x46, 0xD4, 0x66, 0x00, 0x32},
//    {0x28, 0x0A, 0x0C, 0x46, 0xD4, 0x64, 0x12, 0xDE},
// };

// Número de sensores detectados
int nsensor;

void setup() {
 Serial.begin(9600); // Comunicação serial a 9600 bps
 sensor.begin(); // Inicializa os sensores

 // Conta quantos sensores foram encontrados
 nsensor = sensor.getDeviceCount();

 Serial.print("Encontrados ");
 Serial.print(nsensor, DEC);
 Serial.println(" sensores\n");

 // Obtém e armazena o endereço de cada sensor detectado
 for(int i = 0; i < nsensor && i < MAX_SENSOR; i++){
   sensor.getAddress(enderecos[i], i);
 }
}

void loop() {
```

```
 sensor.requestTemperatures(); // Solicita que todos os sensores
façam a leitura da temperatura

 // Percorre os sensores encontrados
 for(int i = 0; i < nsensor && i < MAX_SENSOR; i++){
   float tempC = sensor.getTempC(enderecos[i]); // Lê a temperatura
do sensor pelo endereço

   if(tempC == DEVICE_DISCONNECTED_C){ // Caso o sensor não esteja
respondendo
     Serial.print("Desconectado");
   } else {
     imprimeEndereco(enderecos[i]); // Imprime o endereço do sensor
     Serial.print(":");
     Serial.print(tempC, 1); // Imprime a temperatura com 1 casa
decimal
   }

   // Quebra de linha entre sensores (menos no último)
   if(i < nsensor - 1 && i < MAX_SENSOR - 1){
     Serial.println("");
   }
 }

 Serial.println(""); // Linha em branco após imprimir todos os
sensores
 delay(5000); // Aguarda 5 segundos antes da próxima leitura
}

// Função auxiliar para imprimir o endereço único de cada sensor em
HEX
void imprimeEndereco(DeviceAddress endereco){
 for(uint8_t i = 0; i < 8; i++){
   if(endereco[i] < 16){ // Se o valor for menor que 16, adiciona
um zero à esquerda
     Serial.print("0");
   }
   Serial.print(endereco[i], HEX); // Imprime cada byte em
hexadecimal
 }
 Serial.print("");
}
```

Note que há um outro "**DeviceAddress enderecos[MAX_SENSOR]**" comentado no código, use-o caso queira definir estaticamente os sensores pelos seus endereços. Caso faça isso, algumas partes do código não serão necessárias como a definição máxima de sensores e o loop **for** presente na função **setup()**. Ao final do código é declarada a função para imprimir os endereços dos sensores de temperatura.

5.2 Sensor de Temperatura e Umidade DHT22

A implementação do código desse sensor é mais simples, portanto baixe a biblioteca **DHT sensor library** e implemente-o da seguinte forma:

```
#include <DHT.h>

#define DHTTYPE DHT22 // Sensor DHT22
#define DHTPIN 1 // Pino do Arduino conectado no Sensor(Data)

DHT dht(DHTPIN, DHTTYPE); // Configurando o Sensor DHT - pino e
tipo

void setup(){
 Serial.begin(9600); // Monitor serial 9600 bps
 dht.begin(); // Inicializa a função
}

void loop(){
 delay(1000); // Atraso entre as medições
 float tempDHT = dht.readTemperature(); // Faz a leitura da
Temperatura
 float humDHT = dht.readHumidity(); // Faz a leitura de umidade

 if(isnan(tempDHT)){ // Se algum erro na leitura
   Serial.println("Erro na leitura da Temperatura!");
 } else { // Senão
   Serial.print("Temperatura: "); // Imprime a Temperatura
   Serial.print(tempDHT);
   Serial.println(" *C");
 }
  if(isnan(humDHT)){ // Se algum erro na leitura
   Serial.println("Erro na leitura da Umidade!");
 } else { // Senão
   Serial.print("Umidade: "); // Imprime a Umidade
   Serial.print(humDHT);
```

```
    Serial.println("%");
    Serial.println("");
  }
}
```

5.3 Sensor de Umidade do Solo Capacitivo

Para o sensor de umidade do solo capacitivo, tem-se uma simples implementação do código da seguinte forma:

```
// Número de amostras para o cálculo da média
#define NUMERO_AMOSTRAS 10
// Define a porta analógica para o sensor
#define PINO A0

// Variáveis globais para armazenar valores de leitura e cálculos
int valorUmidade, somatorio, leitura;
// Variável para guardar a média das leituras
float media;

void setup(){
 // Comunicação serial a 9600 bps
 Serial.begin(9600);
 // Configura o pino do sensor como entrada
 pinMode(PINO, INPUT);
}

void loop(){
 somatorio = 0; // Zera o acumulador
 // Faz várias leituras e acumula os valores
 for(int i = 0; i < NUMERO_AMOSTRAS; i++){
   leitura = analogRead(PINO); // Lê o valor analógico do sensor 1
   somatorio += leitura; // Soma ao acumulador
 }
 // Calcula a média das leituras do sensor 1
 media = 1.0 * somatorio / NUMERO_AMOSTRAS;
 Serial.print("Valor1: ");
 Serial.print(media); // Imprime o valor médio lido do sensor 1

 // Converte o valor médio em porcentagem de umidade (0% a 100%)
 // Os valores 546 e 242 são limites calibrados do sensor (solo
seco e molhado, por exemplo)
```

```
 valorUmidade = constrain(map(media, 546, 242, 0, 100), 0, 100);
 Serial.print("  |  ");
 Serial.print("Umidade1: ");
 Serial.print(valorUmidade); // Imprime o valor convertido em
umidade
 Serial.println(" %");

 // Aguarda 1 segundo antes de repetir o processo
 delay(1000);
}
```

Nesse código foi utilizado a média das leituras, que limitadas por amostras definidas no início do código, serve para uma calibração do sensor. Dessa forma, como o sensor é analógico, ou seja, sua leitura vai de 0 a 1023, é feita uma calibragem a partir do valor obtido e converte-o em porcentagem para medir a umidade do solo. Logo, como já dito anteriormente, calibre o sensor como achar necessário.

5.4 Sensor de Umidade do Solo Resistivo

A implementação do código do sensor de umidade do solo resistivo é feito da seguinte forma:

```
// Define a porta analógica para o sensor
#define PINO_SENSOR A0
// Resistor (em ohms) usado no divisor de tensão
#define R1 10000

// Variável para armazenar o valor final em porcentagem de umidade
int valorUmidade;
// Variáveis para cálculos intermediários
float resis_solo, tensao, leitura;

void setup() {
 // Comunicação serial a 9600 bps
 Serial.begin(9600);
 // Configura o pino do sensor como entrada
 pinMode(PINO_SENSOR, INPUT);
}

void loop(){
 // Leitura direta do valor analógico
 Serial.print("Valor analógico: ");
```

```
 Serial.println(analogRead(PINO_SENSOR));
 Serial.println("");

 // Conversão do valor analógico para tensão
 tensao = analogRead(PINO_SENSOR) * 5.0 / 1023.0;
 Serial.print("Tensão: ");
 Serial.print(tensao);
 Serial.println("v");
 Serial.println("");

 // Cálculo da resistência do solo
 resis_solo = (tensao * R1) / (5.0 - tensao);
 Serial.print("Resistividade do solo = ");
 Serial.println(resis_solo); // Valor da resistência equivalente do
solo em ohms
 Serial.println("");
  // Simulação inversa da leitura analógica baseada na resistência
calculada
 leitura = (5.0 * resis_solo) / (R1 + resis_solo); // Recupera a
tensão do divisor
 leitura = leitura * 1023.0 / 5.0; // Converte de volta para o
valor da porta analógica
 Serial.print("Valor da leitura: ");
 Serial.print(leitura);

 // Conversão da leitura em porcentagem de umidade
 // Calibração: solo seco = 0% (1023) e solo molhado = 100% (535)
 valorUmidade = constrain(map(leitura, 1023, 535, 0, 100), 0, 100);
 Serial.print("  |  ");
 Serial.print("Umidade: ");
 Serial.print(valorUmidade); // Imprime a umidade calculada em %
 Serial.println(" %");
 Serial.println("");

 // Aguarda 1 segundo antes da próxima leitura
 delay(1000);
}
```

Agora para uma breve contextualização do código e de como foi feita a calibração do sensor, foi utilizado não só a leitura analógica mas também o cálculo da resistência do solo a partir do divisor de tensão com o resistor fixo R1. Então primeiro é feita a leitura analógica, em seguida a partir da fórmula **Vout = (Leitura *

Vref) / 1023 a leitura é convertida em tensão, que seguindo a fórmula **Vout = (Rsolo / (R1 + Rsolo)) * Vin** e rearranjando-a para **Rsolo = (Vout * R1) / (Vin - Vout)**, é obtida a resistência do solo em ohms. Por fim é feita a reconversão para o valor da leitura usando a segunda fórmula agora com o valor de **Rsolo**, que para concluir é transformado em valor digital e convertido para porcentagem para a medição da umidade do solo.

5.5 Sensor de Luminosidade BH1750

O código para o sensor de luminosidade é bem simples, baixe o biblioteca **BH1750**, e o implemente da seguinte forma:

```
#include <BH1750.h>

BH1750 sensorLux;

void setup(){
 // Inicializa a comunicação serial para envio dos dados ao
computador
 Serial.begin(9600);
 // Inicializa a comunicação com o sensor BH1750
 sensorLux.begin();
}

void loop(){
 // Lê o nível de luminosidade em lux do sensor
 uint16_t luminosidade = sensorLux.readLightLevel();
  Serial.print("Luminosidade: ");
 Serial.print(luminosidade); // Valor em lux
 Serial.println(" lux");
  // Aguarda 1 segundo antes de fazer a próxima leitura
 delay(1000);
}
```

5.6 ESP8266

O ESP8266 é o responsável pela conexão em uma rede wifi e monta um servidor web que retorna os valores dos sensores que podem ser visualizados pelo navegador através de um código HTML simples. Por isso, no código abaixo é utilizado **comandos AT** para a configuração do ESP (estes comandos podem ser visualizados no manual presente no **Tópico 7**), além disso a fim de ser possível o teste do mesmo, no código está presente a leitura do sensor de temperatura

DS18B20, portanto teste o ESP junto desse sensor. Implemente o código da seguinte forma:

```
#include <OneWire.h>
#include <DallasTemperature.h>

#define DEBUG true // Habilita ou desabilita saída de debug
#define MAX_SENSOR 5 // Máximo de sensores DS18B20 que serão
tratados

// Define o barramento OneWire no pino digital 12
OneWire oneWire(12);
// Cria objeto "sensor" que gerencia os DS18B20 conectados ao
barramento
DallasTemperature sensor(&oneWire);
// Array para armazenar os endereços de até MAX_SENSOR sensores
DeviceAddress enderecos[MAX_SENSOR];

int nsensor; // Quantidade de sensores encontrados

void setup(){
 Serial.begin(9600); // Comunicação serial a 9600 bps
 Serial1.begin(115200); // Comunicação com ESP8266 (baud rate
padrão AT)

 // Reseta o módulo
 sendData("AT+RST\r\n", 2000, DEBUG);
 delay(3000);
 // Conecta ao WiFi
 sendData("AT+CWJAP=\"Nome da rede\",\"Senha da rede\"\r\n", 2000,
DEBUG);
 delay(5000);
 // Configura como Station
 sendData("AT+CWMODE=1\r\n", 1000, DEBUG);
 // Mostra IP atribuído
 sendData("AT+CIFSR\r\n", 1000, DEBUG);
 // Habilita múltiplas conexões
 sendData("AT+CIPMUX=1\r\n", 1000, DEBUG);
 // Inicia servidor na porta 80
 sendData("AT+CIPSERVER=1,80\r\n", 1000, DEBUG);

 // Inicializa os sensores de temperatura
```

```
 sensor.begin();
 nsensor = sensor.getDeviceCount(); // Conta quantos sensores estão
no barramento
 Serial.print("Encontrados ");
 Serial.print(nsensor, DEC);
 Serial.println(" sensores\n");

 // Pega os endereços de cada sensor encontrado
 for(int i = 0; i < nsensor && i < MAX_SENSOR; i++){
   sensor.getAddress(enderecos[i], i);
 }
}

void loop(){
 // Verifica se chegou dado do ESP8266
 if(Serial1.available()){
   if(Serial1.find("+IPD,")){ // "+IPD" indica que chegou uma
requisição HTTP
     delay(300);
     sensor.requestTemperatures(); // Solicita atualização de todos
os sensores
     int connectionId = Serial1.read() - 48; // Pega o ID da
conexão (char -> int)

     // Monta página HTML
     String webpage = "<head><meta charset=\"UTF-8\"></head>";
     webpage += "<h1><u>ESP8266 - Sensores de
Temperatura</u></h1>";

     for(int i = 0; i < nsensor && i < MAX_SENSOR; i++){
       float tempC = sensor.getTempC(enderecos[i]);
       if(tempC == DEVICE_DISCONNECTED_C){
         // Caso o sensor não responda
         webpage += "<h2>Sensor ";
         webpage += i + 1;
         webpage += " não encontrado!</h2>";
       } else {
         // Sensor ativo: mostra temperatura e endereço
         webpage += "<h2>Sensor ";
         webpage += i + 1;
         webpage += "(";
         webpage += imprimeEndereco(enderecos[i]);
```

```
        webpage += "): ";
        webpage += String(tempC, 1);
        webpage += " &deg;C</h2>";
      }
    }

    // Envia página ao cliente via ESP8266
    String cipSend = "AT+CIPSEND=";
    cipSend += connectionId;
    cipSend += ",";
    cipSend += webpage.length();
    cipSend += "\r\n";
    sendData(cipSend, 1000, DEBUG);
    sendData(webpage, 1000, DEBUG);

    // Fecha a conexão
    String closeCommand = "AT+CIPCLOSE=";
    closeCommand += connectionId;
    closeCommand += "\r\n";
    sendData(closeCommand, 3000, DEBUG);
  }
 }
}

// Função que envia comandos AT ao ESP8266 e captura resposta
String sendData(String command, const int timeout, boolean debug){
String response = "";
Serial1.print(command);
long int time = millis();
while ((time + timeout) > millis()){
  while (Serial1.available()){
    char c = Serial1.read();
    response += c;
  }
}
if (debug){
  Serial.println(response); // Mostra resposta do ESP8266
}
return response;
}

// Função que formata o endereço de cada sensor DS18B20 em string
```

```
hex
String imprimeEndereco(DeviceAddress endereco){
String sEndereco = "";
for(uint8_t i = 0; i < 8; i++){
  if(endereco[i] < 16){
    sEndereco += "0"; // Adiciona zero à esquerda se < 16 (para
manter 2 dígitos)
  }
  sEndereco += String(endereco[i], HEX);
}
sEndereco.toUpperCase(); // Deixa em maiúsculo
return sEndereco;
}
```

Ao compilar e executar o código, no terminal serial será impresso o endereço IP na qual o ESP está conectado, digite o endereço no seu navegador e deve aparecer os valores de leitura dos sensores.

5.7 Código completo

Por último tem-se o código completo do projeto, este reúne todas as bibliotecas utilizadas, assim como os trechos de códigos mostrados anteriormente, funcionando de forma integrada. Além disso, é introduzida uma nova biblioteca chamada **ArduinoJson**, que deve ser baixada. Essa biblioteca diz respeito a parte do código onde além da requisição HTTP tradicional feita pelo módulo, o sistema também disponibiliza um arquivo no formato **JSON** com todas as leituras dos sensores. Esse formato é utilizado pelo script do coletor dos dados dos sensores para uma melhor leitura e manipulação dos mesmos. A implementação é feita da seguinte forma:

```
#include <OneWire.h>
#include <DallasTemperature.h>
#include <ArduinoJson.h>
#include <DHT.h>
#include <BH1750.h>

// ==================== DEFINIÇÕES ====================
#define NUMERO_AMOSTRAS 10 // Número de amostras para média
#define DEBUG true // Ativa/desativa mensagens de debug
#define MAX_SENSOR 5 // Número máximo de sensores DS18B20
suportados
#define DHTTYPE DHT22 // Tipo de sensor DHT usado
```

```
#define DHTPIN1 7 // Pino do primeiro sensor DHT22
#define DHTPIN2 5 // Pino do segundo sensor DHT22
#define R1 10000 // Resistor de 10kΩ usado no divisor de tensão
// Pinos analógicos para os sensores de umidade do solo capacitivo
#define CAPACIPIN1 A0
#define CAPACIPIN2 A1
#define CAPACIPIN3 A5
#define CAPACIPIN4 A6
#define CAPACIPIN5 A7
#define CAPACIPIN6 A8
#define CAPACIPIN7 A9
#define CAPACIPIN8 A10
#define CAPACIPIN9 A11
#define CAPACIPIN10 A12
// Pinos analógicos para os sensores de umidade do solo resistivo
#define RESISTPIN1 A2
#define RESISTPIN2 A3
#define RESISTPIN3 A4

// ==================== OBJETOS DOS SENSORES ====================
OneWire oneWire(12); // Comunicação OneWire no pino 12
DallasTemperature sensor(&oneWire); // Objeto de leitura DS18B20
DeviceAddress enderecos[MAX_SENSOR]; // Vetor para armazenar
endereços dos sensores DS18B20
BH1750 sensorLux; // Objeto para sensor de luminosidade
DHT dht1(DHTPIN1, DHTTYPE); // Objeto do primeiro sensor DHT22
DHT dht2(DHTPIN2, DHTTYPE); // Objeto do segundo sensor DHT22

// ==================== VARIÁVEIS GLOBAIS ====================
int nsensor, somatorio;
float media, leitura, resis_solo, tensao;
int umidadeCapaci;
int umidadeResist;
uint16_t luminosidade;


// ==================== SETUP ====================
void setup(){
 Serial.begin(9600); // Comunicação serial a 9600 bps
 Serial1.begin(115200); // Comunicação com ESP8266 (baud rate
padrão AT)
  // Reinicia o módulo e espera resposta completa
```

```
 sendData("AT+RST\r\n", 3000, DEBUG);
 delay(5000);

 // Verifica firmware
 sendData("AT+GMR\r\n", 1000, DEBUG);

 // Modo estação
 sendData("AT+CWMODE=1\r\n", 2000, DEBUG);
 delay(2000);

 // Conecta Wi-Fi
 sendData("AT+CWJAP=\"Nome da rede\",\"Senha da rede\"\r\n", 5000,
DEBUG);
 delay(8000);

 // Mostra IP obtido
 sendData("AT+CIFSR\r\n", 2000, DEBUG);
 delay(1000);

 // Habilita múltiplas conexões
 sendData("AT+CIPMUX=1\r\n", 1000, DEBUG);
 delay(1000);

 // Inicia servidor na porta 80 (HTTP)
 sendData("AT+CIPSERVER=1,80\r\n", 1000, DEBUG);
 delay(1000);

 // Inicialização dos sensores
 sensor.begin();
 sensorLux.begin();
 dht1.begin();
 dht2.begin();

 // Configuração dos pinos como entrada
 pinMode(CAPACIPIN1, INPUT);
 pinMode(CAPACIPIN2, INPUT);
 pinMode(CAPACIPIN3, INPUT);
 pinMode(CAPACIPIN4, INPUT);
 pinMode(CAPACIPIN5, INPUT);
 pinMode(CAPACIPIN6, INPUT);
 pinMode(CAPACIPIN7, INPUT);
 pinMode(CAPACIPIN8, INPUT);
```

```
 pinMode(CAPACIPIN9, INPUT);
 pinMode(CAPACIPIN10, INPUT);
 pinMode(RESISTPIN1, INPUT);
 pinMode(RESISTPIN2, INPUT);
 pinMode(RESISTPIN3, INPUT);

 // Descobre quantos sensores DS18B20 estão conectados
 nsensor = sensor.getDeviceCount();
 for(int i = 0; i < nsensor && i < MAX_SENSOR; i++){
   sensor.getAddress(enderecos[i], i); // Salva endereço de cada
sensor
 }
}

// ==================== LOOP PRINCIPAL ====================
void loop(){
 // Verifica se o ESP8266 recebeu uma requisição
 if(Serial1.available()){
   if(Serial1.find("+IPD,")){  // Identifica início da requisição
     delay(300);
     int connectionId = Serial1.read() - 48; // ID da conexão

     // Limpa o buffer até encontrar os dados da requisição
     while(Serial1.available() && Serial1.read() != ':') {}

     // Lê a linha da requisição HTTP
     String request = Serial1.readStringUntil('\r');

     // Solicita atualização das temperaturas DS18B20
     sensor.requestTemperatures();

     String response;
     if(request.indexOf("GET /html") != -1){
       // ------------------ RESPOSTA HTML ------------------
       response = "HTTP/1.1 200 OK\r\n";
       response += "Content-Type: text/html\r\n";
       response += "Connection: close\r\n\r\n";
       response += "<!DOCTYPE html><html>";
       response += "<head><meta charset=\"UTF-8\"></head>";
       response += "<body><h1><u>ESP8266 - Sensores da
estufa</u></h1>";
```

```
        // Mostra hora da leitura
        response += "<h3 id='hora'></h3>";
        response += "<script>";
        response += "const agora = new Date();";
        response += "document.getElementById('hora').innerText = ";
        response += "\"Data/Hora da leitura: \" +
agora.toLocaleString();";
        response += "</script>";
        response += "<hr>";

        // --- Temperaturas DS18B20 ---
        response += "<h3><u>Sensores de Temperatura</u></h3>";
        for(int i = 0; i < nsensor && i < MAX_SENSOR; i++) {
          float tempC = sensor.getTempC(enderecos[i]);
          if(tempC == DEVICE_DISCONNECTED_C) {
            response += "<h2>Sensor " + String(i+1) + " não
encontrado!</h2>";
          } else {
            response += "<h2>Sensor " + String(i+1) + " (" +
imprimeEndereco(enderecos[i]) + "): ";
            response += String(tempC, 1) + " &deg;C</h2>";
          }
        }
        response += "<hr>";

        // --- DHT22 ---
        response += "<h3><u>Sensores de umidade do ar e
temperatura</u></h3>";
        float tempDHT1 = dht1.readTemperature();
        float humDHT1 = dht1.readHumidity();
        float tempDHT2 = dht2.readTemperature();
        float humDHT2 = dht2.readHumidity();

        if(isnan(humDHT1)){
          response += "<h2>Sensor 1 desconectado!</h2>";
        } else {
          response += "<h2>Umidade: " + String(humDHT1) + "% | ";
          response += "Temperatura: " + String(tempDHT1) + "
&deg;C</h2>";
        }

        if(isnan(humDHT2)){
```

```
        response += "<h2>Sensor 2 desconectado!</h2>";
      } else {
        response += "<h2>Umidade: " + String(humDHT2) + "% | ";
        response += "Temperatura: " + String(tempDHT2) + "
&deg;C</h2>";
      }
      response += "<hr>";

      // --- Umidade do solo capacitiva ---
      response += "<h3><u>Sensores de umidade do solo
capacitivo</u></h3>";
      somatorio = 0;
      for(int i = 0; i < NUMERO_AMOSTRAS; i++){
        umidadeCapaci = analogRead(CAPACIPIN1);
        somatorio += umidadeCapaci;
      }
      media = 1.0 * somatorio / NUMERO_AMOSTRAS;
      response += "<h2>Valor: " + String(media) + "g/m³";
      umidadeCapaci = constrain(map(media, 546, 242, 0, 100), 0,
100);
      response += " | Umidade: " + String(umidadeCapaci) +
"%</h2>";

      somatorio = 0;
      for(int i = 0; i < NUMERO_AMOSTRAS; i++){
        umidadeCapaci = analogRead(CAPACIPIN2);
        somatorio += umidadeCapaci;
      }
      media = 1.0 * somatorio / NUMERO_AMOSTRAS;
      response += "<h2>Valor: " + String(media) + "g/m³";
      umidadeCapaci = constrain(map(media, 546, 242, 0, 100), 0,
100);
      response += " | Umidade: " + String(umidadeCapaci) +
"%</h2>";

      somatorio = 0;
      for(int i = 0; i < NUMERO_AMOSTRAS; i++){
        umidadeCapaci = analogRead(CAPACIPIN3);
        somatorio += umidadeCapaci;
      }
      media = 1.0 * somatorio / NUMERO_AMOSTRAS;
      response += "<h2>Valor: " + String(media) + "g/m³";
```

```
        umidadeCapaci = constrain(map(media, 546, 242, 0, 100), 0,
100);
        response += " | Umidade: " + String(umidadeCapaci) +
"%</h2>";

        somatorio = 0;
        for(int i = 0; i < NUMERO_AMOSTRAS; i++){
          umidadeCapaci = analogRead(CAPACIPIN4);
          somatorio += umidadeCapaci;
        }
        media = 1.0 * somatorio / NUMERO_AMOSTRAS;
        response += "<h2>Valor: " + String(media) + "g/m³";
        umidadeCapaci = constrain(map(media, 546, 242, 0, 100), 0,
100);
        response += " | Umidade: " + String(umidadeCapaci) +
"%</h2>";

        somatorio = 0;
        for(int i = 0; i < NUMERO_AMOSTRAS; i++){
          umidadeCapaci = analogRead(CAPACIPIN5);
          somatorio += umidadeCapaci;
        }
        media = 1.0 * somatorio / NUMERO_AMOSTRAS;
        response += "<h2>Valor: " + String(media) + "g/m³";
        umidadeCapaci = constrain(map(media, 546, 242, 0, 100), 0,
100);
        response += " | Umidade: " + String(umidadeCapaci) +
"%</h2>";

        somatorio = 0;
        for(int i = 0; i < NUMERO_AMOSTRAS; i++){
          umidadeCapaci = analogRead(CAPACIPIN6);
          somatorio += umidadeCapaci;
        }
        media = 1.0 * somatorio / NUMERO_AMOSTRAS;
        response += "<h2>Valor: " + String(media) + "g/m³";
        umidadeCapaci = constrain(map(media, 546, 242, 0, 100), 0,
100);
        response += " | Umidade: " + String(umidadeCapaci) +
"%</h2>";

        somatorio = 0;
```

```
      for(int i = 0; i < NUMERO_AMOSTRAS; i++){
        umidadeCapaci = analogRead(CAPACIPIN7);
        somatorio += umidadeCapaci;
      }
      media = 1.0 * somatorio / NUMERO_AMOSTRAS;
      response += "<h2>Valor: " + String(media) + "g/m³";
      umidadeCapaci = constrain(map(media, 546, 242, 0, 100), 0,
100);
      response += " | Umidade: " + String(umidadeCapaci) +
"%</h2>";

      somatorio = 0;
      for(int i = 0; i < NUMERO_AMOSTRAS; i++){
        umidadeCapaci = analogRead(CAPACIPIN8);
        somatorio += umidadeCapaci;
      }
      media = 1.0 * somatorio / NUMERO_AMOSTRAS;
      response += "<h2>Valor: " + String(media) + "g/m³";
      umidadeCapaci = constrain(map(media, 546, 242, 0, 100), 0,
100);
      response += " | Umidade: " + String(umidadeCapaci) +
"%</h2>";

      somatorio = 0;
      for(int i = 0; i < NUMERO_AMOSTRAS; i++){
        umidadeCapaci = analogRead(CAPACIPIN9);
        somatorio += umidadeCapaci;
      }
      media = 1.0 * somatorio / NUMERO_AMOSTRAS;
      response += "<h2>Valor: " + String(media) + "g/m³";
      umidadeCapaci = constrain(map(media, 546, 242, 0, 100), 0,
100);
      response += " | Umidade: " + String(umidadeCapaci) +
"%</h2>";

      somatorio = 0;
      for(int i = 0; i < NUMERO_AMOSTRAS; i++){
        umidadeCapaci = analogRead(CAPACIPIN10);
        somatorio += umidadeCapaci;
      }
      media = 1.0 * somatorio / NUMERO_AMOSTRAS;
      response += "<h2>Valor: " + String(media) + "g/m³";
```

```
        umidadeCapaci = constrain(map(media, 546, 242, 0, 100), 0,
100);
        response += " | Umidade: " + String(umidadeCapaci) +
"%</h2>";

        // --- Umidade do solo resistiva ---
        response += "<h3><u>Sensor de umidade do solo
resistivo</u></h3>";
        tensao = analogRead(RESISTPIN1) * 5.0 / 1023.0;
        resis_solo = (tensao * R1) / (5.0 - tensao);
        leitura = (5.0 * resis_solo) / (R1 + resis_solo);
        leitura = leitura * 1023.0 / 5.0;
        response += "<h2>Valor: " + String(leitura) + "g/m³";
        umidadeResist = constrain(map(leitura, 1023, 535, 0, 100),
0, 100);
        response += " | Umidade: " + String(umidadeResist) +
"%</h2>";

        tensao = analogRead(RESISTPIN2) * 5.0 / 1023.0;
        resis_solo = (tensao * R1) / (5.0 - tensao);
        leitura = (5.0 * resis_solo) / (R1 + resis_solo);
        leitura = leitura * 1023.0 / 5.0;
        response += "<h2>Valor: " + String(leitura) + "g/m³";
        umidadeResist = constrain(map(leitura, 1023, 535, 0, 100),
0, 100);
        response += " | Umidade: " + String(umidadeResist) +
"%</h2>";

        tensao = analogRead(RESISTPIN3) * 5.0 / 1023.0;
        resis_solo = (tensao * R1) / (5.0 - tensao);
        leitura = (5.0 * resis_solo) / (R1 + resis_solo);
        leitura = leitura * 1023.0 / 5.0;
        response += "<h2>Valor: " + String(leitura) + "g/m³";
        umidadeResist = constrain(map(leitura, 1023, 535, 0, 100),
0, 100);
        response += " | Umidade: " + String(umidadeResist) +
"%</h2>";
        response += "<hr>";

        // --- Sensor de luminosidade BH1750 ---
        response += "<h3><u>Sensor de luminosidade</u></h3>";
        luminosidade = sensorLux.readLightLevel();
```

```
        if(isnan(luminosidade) || luminosidade < 0 || luminosidade >
10000){
            response += "<h2>Sensor de luminosidade
desconectado!</h2>";
        } else {
            response += "<h2>Luminosidade: " + String(luminosidade)
+ " lux</h2>";
        }

        response += "</body></html>";
      } else {
        // ------------------ RESPOSTA JSON ------------------
        StaticJsonDocument<512> jsonDoc;
        JsonArray sensors = jsonDoc.createNestedArray("sensors");

        // Preenche JSON com os dados dos sensores
        // Cada leitura é organizada em objetos JSON com id, tipo,
valor e unidade
        int sensorID = 1;

        for(int i = 0; i < nsensor && i < MAX_SENSOR; i++) {
          float tempC = sensor.getTempC(enderecos[i]);
          if(tempC != DEVICE_DISCONNECTED_C) {
            JsonObject tempData = sensors.createNestedObject();
            tempData["id"] = sensorID++;
            tempData["type"] = "DS18B20";
            tempData["value"] = tempC;
            tempData["unit"] = "C";
          }
        }

        float tempDHT1 = dht1.readTemperature();
        float humDHT1 = dht1.readHumidity();
        float tempDHT2 = dht2.readTemperature();
        float humDHT2 = dht2.readHumidity();

        JsonObject dht1Data = sensors.createNestedObject();
        dht1Data["id"] = sensorID++;
        dht1Data["type"] = "DHT22";
        dht1Data["value"] = humDHT1;
        dht1Data["unit"] = "%";
```

```
        JsonObject dht2Data = sensors.createNestedObject();
        dht2Data["id"] = sensorID++;
        dht2Data["type"] = "DHT22";
        dht2Data["value"] = humDHT2;
        dht2Data["unit"] = "%";

        somatorio = 0;
        for(int i = 0; i < NUMERO_AMOSTRAS; i++){
          umidadeResist = analogRead(CAPACIPIN1);
          somatorio += umidadeResist;
        }
        media = 1.0 * somatorio / NUMERO_AMOSTRAS;
        JsonObject capacitivo1 = sensors.createNestedObject();
        capacitivo1["id"] = sensorID++;
        capacitivo1["type"] = "capacitivo";
        capacitivo1["value"] = constrain(map(media, 546, 242, 0,
100), 0, 100);
        capacitivo1["unit"] = "%";

        somatorio = 0;
        for(int i = 0; i < NUMERO_AMOSTRAS; i++){
          umidadeResist = analogRead(CAPACIPIN2);
          somatorio += umidadeResist;
        }
        media = 1.0 * somatorio / NUMERO_AMOSTRAS;
        JsonObject capacitivo2 = sensors.createNestedObject();
        capacitivo2["id"] = sensorID++;
        capacitivo2["type"] = "capacitivo";
        capacitivo2["value"] = constrain(map(media, 546, 242, 0,
100), 0, 100);
        capacitivo2["unit"] = "%";

        somatorio = 0;
        for(int i = 0; i < NUMERO_AMOSTRAS; i++){
          umidadeResist = analogRead(CAPACIPIN3);
          somatorio += umidadeResist;
        }
        media = 1.0 * somatorio / NUMERO_AMOSTRAS;
        JsonObject capacitivo3 = sensors.createNestedObject();
        capacitivo3["id"] = sensorID++;
        capacitivo3["type"] = "capacitivo";
        capacitivo3["value"] = constrain(map(media, 546, 242, 0,
```

```
100), 0, 100);
      capacitivo3["unit"] = "%";

      somatorio = 0;
      for(int i = 0; i < NUMERO_AMOSTRAS; i++){
        umidadeResist = analogRead(CAPACIPIN4);
        somatorio += umidadeResist;
      }
      media = 1.0 * somatorio / NUMERO_AMOSTRAS;
      JsonObject capacitivo4 = sensors.createNestedObject();
      capacitivo4["id"] = sensorID++;
      capacitivo4["type"] = "capacitivo";
      capacitivo4["value"] = constrain(map(media, 546, 242, 0,
100), 0, 100);
      capacitivo4["unit"] = "%";

      somatorio = 0;
      for(int i = 0; i < NUMERO_AMOSTRAS; i++){
        umidadeResist = analogRead(CAPACIPIN5);
        somatorio += umidadeResist;
      }
      media = 1.0 * somatorio / NUMERO_AMOSTRAS;
      JsonObject capacitivo5 = sensors.createNestedObject();
      capacitivo5["id"] = sensorID++;
      capacitivo5["type"] = "capacitivo";
      capacitivo5["value"] = constrain(map(media, 546, 242, 0,
100), 0, 100);
      capacitivo5["unit"] = "%";

      somatorio = 0;
      for(int i = 0; i < NUMERO_AMOSTRAS; i++){
        umidadeResist = analogRead(CAPACIPIN6);
        somatorio += umidadeResist;
      }
      media = 1.0 * somatorio / NUMERO_AMOSTRAS;
      JsonObject capacitivo6 = sensors.createNestedObject();
      capacitivo6["id"] = sensorID++;
      capacitivo6["type"] = "capacitivo";
      capacitivo6["value"] = constrain(map(media, 546, 242, 0,
100), 0, 100);
      capacitivo6["unit"] = "%";
```

```
        somatorio = 0;
        for(int i = 0; i < NUMERO_AMOSTRAS; i++){
          umidadeResist = analogRead(CAPACIPIN7);
          somatorio += umidadeResist;
        }
        media = 1.0 * somatorio / NUMERO_AMOSTRAS;
        JsonObject capacitivo7 = sensors.createNestedObject();
        capacitivo7["id"] = sensorID++;
        capacitivo7["type"] = "capacitivo";
        capacitivo7["value"] = constrain(map(media, 546, 242, 0,
100), 0, 100);
        capacitivo7["unit"] = "%";

        somatorio = 0;
        for(int i = 0; i < NUMERO_AMOSTRAS; i++){
          umidadeResist = analogRead(CAPACIPIN8);
          somatorio += umidadeResist;
        }
        media = 1.0 * somatorio / NUMERO_AMOSTRAS;
        JsonObject capacitivo8 = sensors.createNestedObject();
        capacitivo8["id"] = sensorID++;
        capacitivo8["type"] = "capacitivo";
        capacitivo8["value"] = constrain(map(media, 546, 242, 0,
100), 0, 100);
        capacitivo8["unit"] = "%";

        somatorio = 0;
        for(int i = 0; i < NUMERO_AMOSTRAS; i++){
          umidadeResist = analogRead(CAPACIPIN9);
          somatorio += umidadeResist;
        }
        media = 1.0 * somatorio / NUMERO_AMOSTRAS;
        JsonObject capacitivo9 = sensors.createNestedObject();
(DS18B20, DHT22, capacitivos, resistivos, luminosidade)
        capacitivo9["id"] = sensorID++;
        capacitivo9["type"] = "capacitivo";
        capacitivo9["value"] = constrain(map(media, 546, 242, 0,
100), 0, 100);
        capacitivo9["unit"] = "%";

        somatorio = 0;
        for(int i = 0; i < NUMERO_AMOSTRAS; i++){
```

```
        umidadeResist = analogRead(CAPACIPIN10);
        somatorio += umidadeResist;
      }
      media = 1.0 * somatorio / NUMERO_AMOSTRAS;
      JsonObject capacitivo10 = sensors.createNestedObject();
      capacitivo10["id"] = sensorID++;
      capacitivo10["type"] = "capacitivo";
      capacitivo10["value"] = constrain(map(media, 546, 242, 0,
100), 0, 100);
      capacitivo10["unit"] = "%";

      tensao = analogRead(RESISTPIN1) * 5.0 / 1023.0;
      resis_solo = (tensao * R1) / (5.0 - tensao);
      leitura = (5.0 * resis_solo) / (R1 + resis_solo);
      leitura = leitura * 1023.0 / 5.0;
      JsonObject resistivo1 = sensors.createNestedObject();
      resistivo1["id"] = sensorID++;
      resistivo1["type"] = "resistivo";
      resistivo1["value"] = constrain(map(leitura, 1023, 515, 0,
100), 0, 100);
      resistivo1["unit"] = "%";

      tensao = analogRead(RESISTPIN2) * 5.0 / 1023.0;
      resis_solo = (tensao * R1) / (5.0 - tensao);
      leitura = (5.0 * resis_solo) / (R1 + resis_solo);
      leitura = leitura * 1023.0 / 5.0;
      JsonObject resistivo2 = sensors.createNestedObject();
      resistivo2["id"] = sensorID++;
      resistivo2["type"] = "resistivo";
      resistivo2["value"] = constrain(map(leitura, 1023, 515, 0,
100), 0, 100);
      resistivo2["unit"] = "%";

      tensao = analogRead(RESISTPIN3) * 5.0 / 1023.0;
      resis_solo = (tensao * R1) / (5.0 - tensao);
      leitura = (5.0 * resis_solo) / (R1 + resis_solo);
      leitura = leitura * 1023.0 / 5.0;
      JsonObject resistivo3 = sensors.createNestedObject();
      resistivo3["id"] = sensorID++;
      resistivo3["type"] = "resistivo";
      resistivo3["value"] = constrain(map(leitura, 1023, 515, 0,
100), 0, 100);
```

```
        resistivo3["unit"] = "%";

        luminosidade = sensorLux.readLightLevel();
        JsonObject luxData = sensors.createNestedObject();
        luxData["id"] = sensorID++;
        luxData["type"] = "BH1750";
        luxData["value"] = luminosidade;
        luxData["unit"] = "lux";

        // Converte JSON em string e prepara resposta HTTP
        String jsonResponse;
        serializeJson(jsonDoc, jsonResponse);
        response = "HTTP/1.1 200 OK\r\n";
        response += "Content-Type: application/json\r\n";
        response += "Connection: close\r\n\r\n";
        response += jsonResponse;
      }

      // Envia a resposta
      String cipSend = "AT+CIPSEND=" + String(connectionId) + "," +
String(response.length()) + "\r\n";
      sendData(cipSend, 1000, DEBUG);
      sendData(response, 1000, DEBUG);

      // Fecha a conexão
      sendData("AT+CIPCLOSE=" + String(connectionId) + "\r\n", 1000,
DEBUG);
    }
  }
}

// ==================== FUNÇÕES AUXILIARES ====================
// Envia comando AT ao ESP8266 e retorna resposta
String sendData(String command, const int timeout, boolean debug) {
  String response = "";
  Serial1.print(command);
  long int time = millis();
  while((time + timeout) > millis()) {
    while(Serial1.available()) {
      char c = Serial1.read();
      response += c;
    }
```

```
  }
  if(debug) Serial.println(response);
  return response;
}

// Converte endereço do DS18B20 em string hex
String imprimeEndereco(DeviceAddress endereco) {
  String sEndereco = "";
  for(uint8_t i = 0; i < 8; i++) {
    if(endereco[i] < 16) sEndereco += "0";
    sEndereco += String(endereco[i], HEX);
  }
  sEndereco.toUpperCase();
  return sEndereco;
}
```

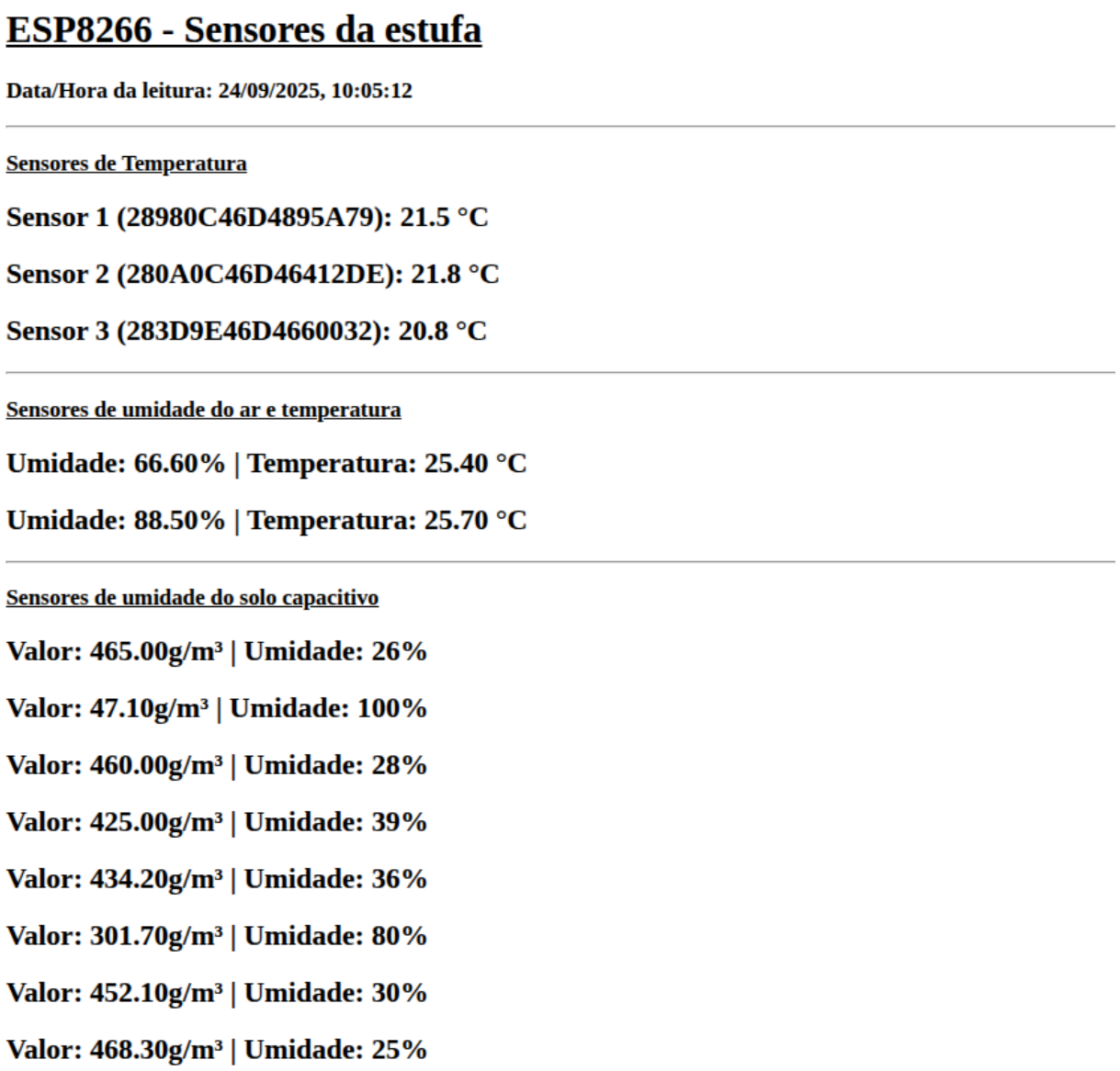
Atenção, pois o JSON precisa seguir a estrutura que está no código para que o coletor possa funcionar corretamente. A estrutura tem que ser:

```
{
   "sensors":
      "id":
      "type":
      "value":
      "unit":
}
```

5.8 Resultado esperado

Agora com toda a montagem pronta e com o código completo em mãos, compile e carregue o código para o **Arduíno Mega** e utilize uma fonte de 9V para ligar o arduino para fazer uso do monitoramento no ambiente que desejar.

No terminal serial deve aparecer o endereço IP da rede conectada, com isso digite o endereço no navegador (utilize “/html” após o endereço para retornar a página, qualquer outra rota retorna o JSON) para visualizar os valores das leituras dos sensores. O resultado esperado é uma página web que deve se parecer com isso:



ESP8266 - Sensores da estufa

Data/Hora da leitura: 24/09/2025, 10:05:12

Sensores de Temperatura

Sensor 1 (28980C46D4895A79): 21.5 °C

Sensor 2 (280A0C46D46412DE): 21.8 °C

Sensor 3 (283D9E46D4660032): 20.8 °C

Sensores de umidade do ar e temperatura

Umidade: 66.60% | Temperatura: 25.40 °C

Umidade: 88.50% | Temperatura: 25.70 °C

Sensores de umidade do solo capacitivo

Valor: 465.00g/m³ | Umidade: 26%

Valor: 47.10g/m³ | Umidade: 100%

Valor: 460.00g/m³ | Umidade: 28%

Valor: 425.00g/m³ | Umidade: 39%

Valor: 434.20g/m³ | Umidade: 36%

Valor: 301.70g/m³ | Umidade: 80%

Valor: 452.10g/m³ | Umidade: 30%

Valor: 468.30g/m³ | Umidade: 25%

Figura 5.1

Na página pode ser observado, além dos valores dos sensores, a data e o horário na qual a leitura foi feita, isso é bem útil para um controle e administração do sistema de monitoramento.

Verifique a página, caso alguns sensores apresentarem valores divergentes do esperado é devido à necessidade de uma calibração mais precisa para garantir valores mais confiáveis. Nesse cenário tente alguma técnica para a calibração dos sensores.

6. Execução do coletor de dados e Dashboard dos sensores

Com os sensores e o servidor web funcionando até aqui, vamos configurar e executar o ambiente para coleta das leituras dos sensores, onde os dados serão

salvos em arquivos **CSV** e caso ocorra algum erro com a coleta e/ou o servidor, um alerta por email é disparado para a noção do problema.

6.1 Gerar senha de aplicativo do Gmail

Para receber um alerta pelo Gmail de qualquer problema que houver com a coleta dos dados, que pode ir desde demora na resposta do servidor a erros de conexão, é necessário uma senha de aplicativo para autenticação no envio do email. Gerar essa senha é simples, segue o passo a passo:

1. No Google pesquise **"google conta"** ou **"gerenciar sua conta do google"**, clique na primeira pesquisa e deve aparecer a seguinte tela**:**

figura 6.1

Clique em **"Acessar a conta do Google"** e entre na sua conta.

2. Na tela inicial no campo de pesquisa digite **"senhas de app"**, vá em **"Senhas de app":**

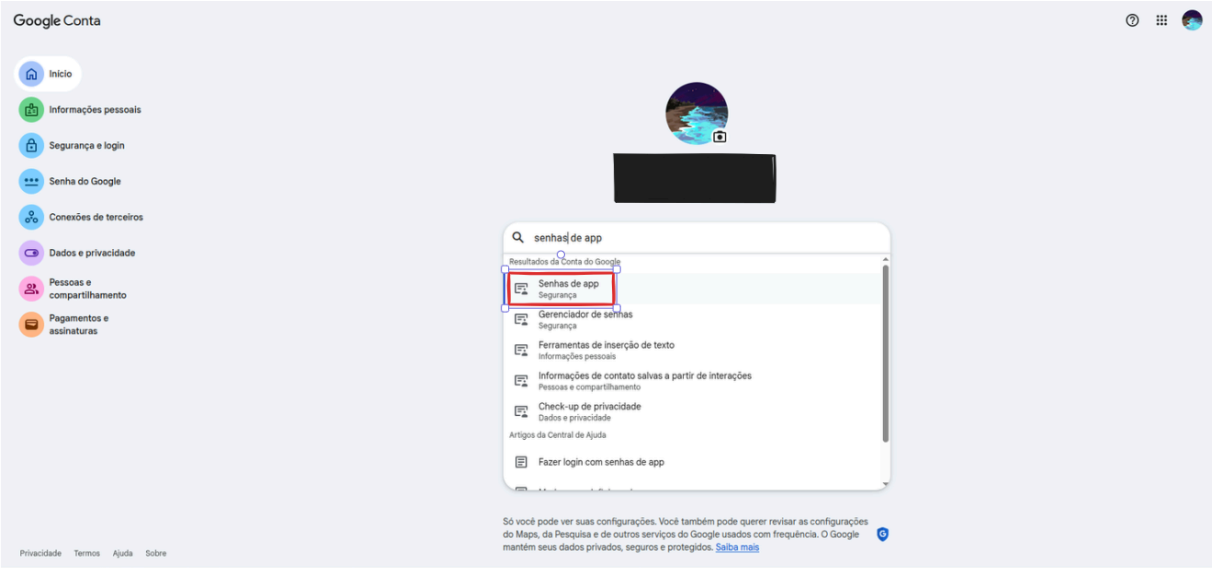


figura 6.2

3. Será necessário autenticar na sua conta mais uma vez, depois disso feito a seguinte tela aparecerá:

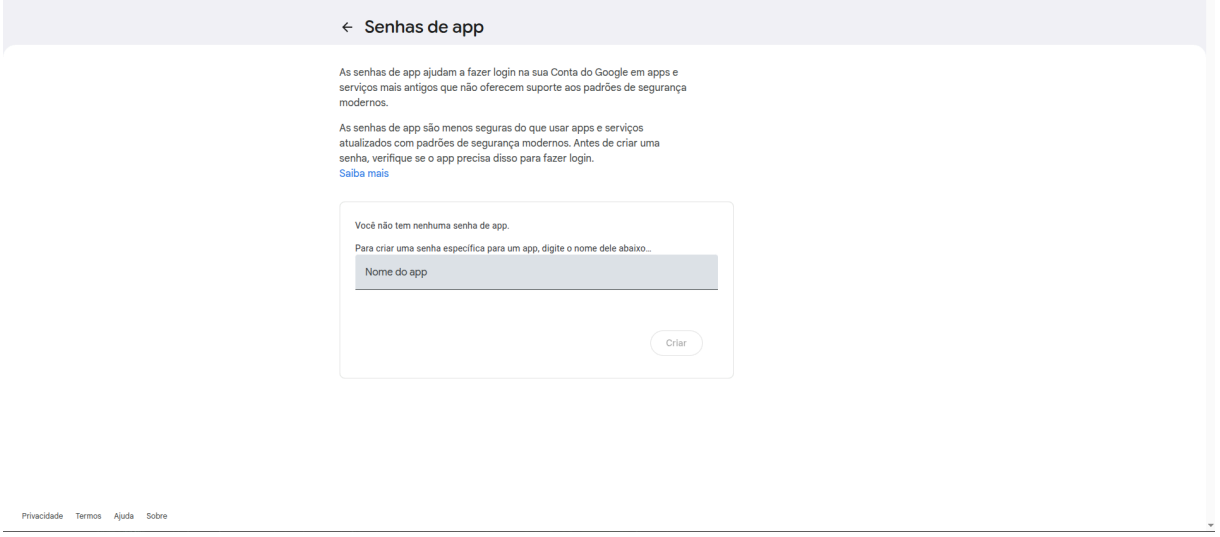


figura 6.3

Digite o nome que quiser para sua senha de aplicativo no campo que aparece, depois clique em **"Criar"** e sua senha de aplicativo deve aparecer. **Agora o mais importante**, copie e cole essa senha, guarde-a em um local seguro, pois a senha não aparece de novo, e não compartilhe essa senha com ninguém.

6.2 Executando o instalador do script

Agora é hora de executar o script para coleta dos dados, para isso você terá quatro arquivos:

- Dashboard.py

- Coletor.py
- requirements.txt
- installer.sh

Crie uma pasta no computador com o nome que desejar, evite acentos e espaços, utilize caracteres como o **underscore(_)** e/ou **traço(-)**, um bom exemplo seria os nomes **Coletor** ou **Coletor_sensores**. Depois disso, copie os três arquivos e cole-os na pasta criada. Com o gerenciador de arquivo aberto e na pasta na qual foi criada, clique como botão direito do mouse em qualquer lugar e selecione **"Abrir no terminal"** como mostra a figura 6.4:

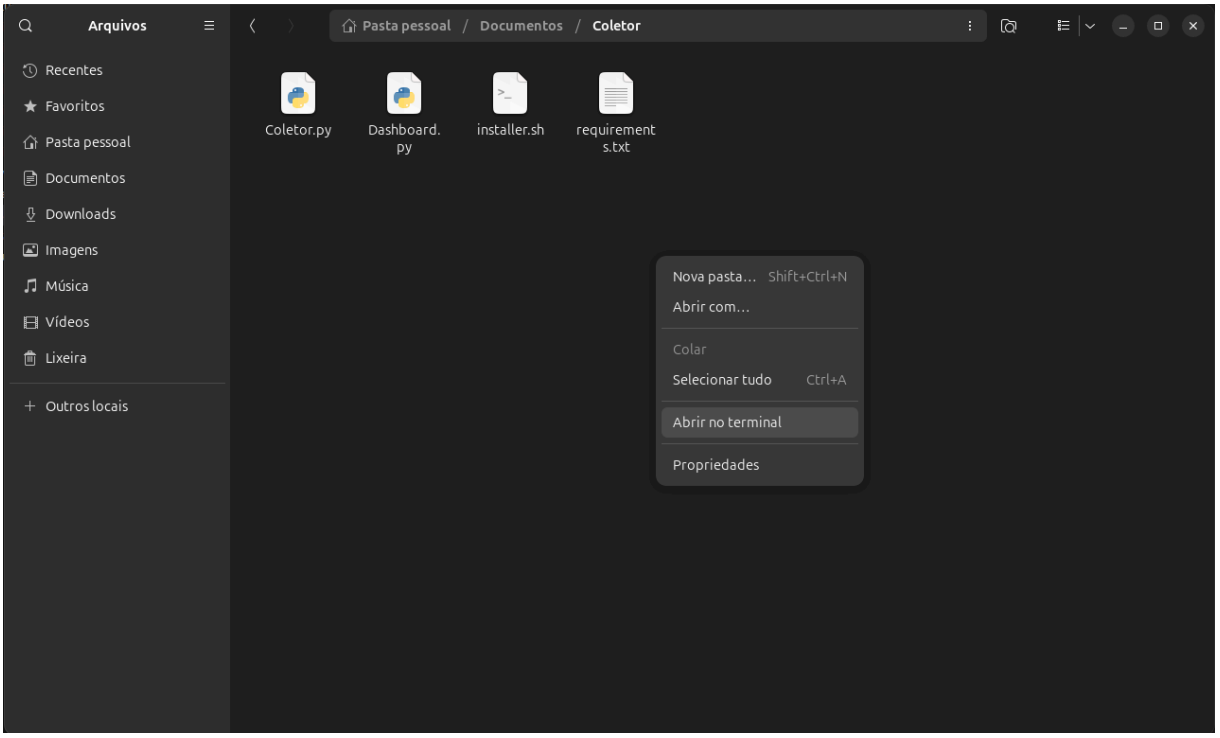


figura 6.4

Um terminal Linux deve abrir, e nele digite o seguinte comando:

```
chmod +x installer.sh
```

Depois disso feito é hora de executar o instalador para o coletor, digite o comando:

```
./installer.sh
```

Assim o instalador começará a configurar o ambiente para a coleta. No terminal será necessário seguir algumas instruções e inserir algumas informações como:

- O IP do servidor web no qual o ESP8266 está rodando;
- Seu Email e sua senha de aplicativo que será o remetente dos emails de alerta;
- Quantidade de leituras que deseja que o script realize;
- Qual o intervalo entre cada leitura;
- Quantidade de tentativas de conexão antes de enviar o alerta;
- Emails adicionais caso queira que o alerta seja enviado para mais pessoas (esta etapa é opcional).

Depois que todas essas informações foram inseridas o instalador deve finalizar com a mensagem **“+++ Instalação concluída com sucesso! +++”.** A pasta do coletor deve se parecer com isso:

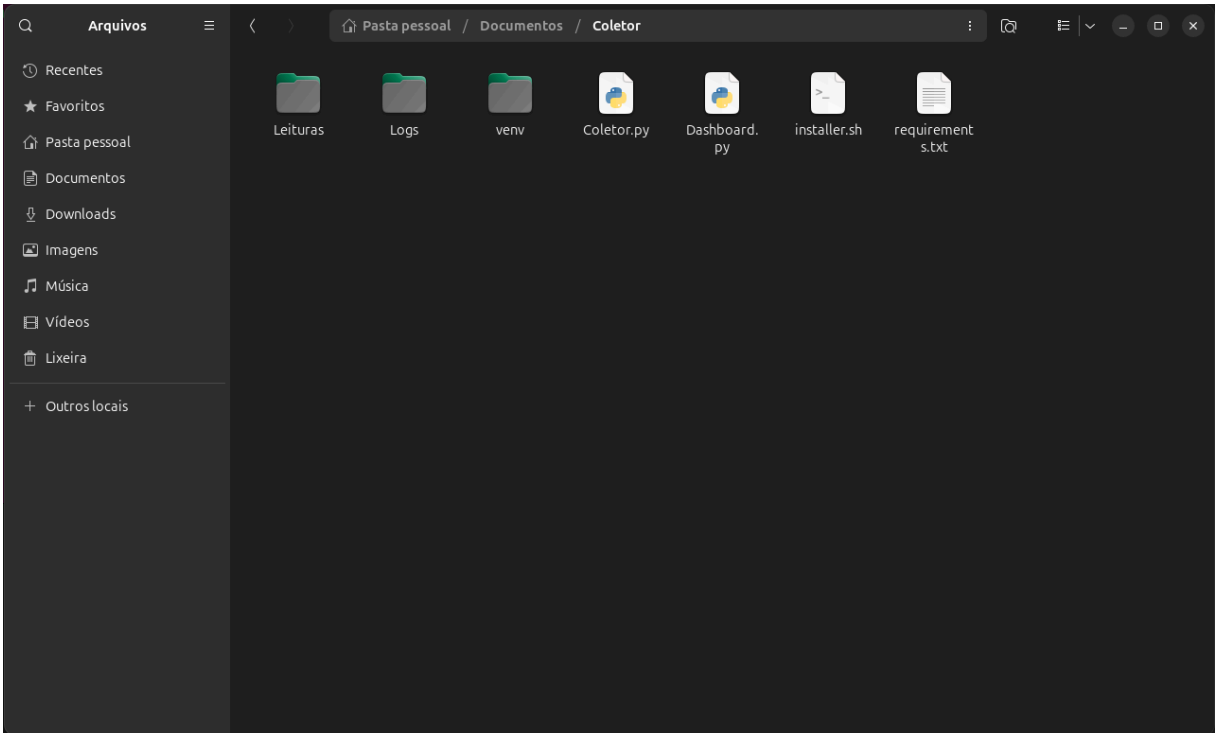


figura 6.5

Aqui foram criados três pastas adicionais:

- O **venv**, que é o ambiente virtual no Python no qual foram instalados as bibliotecas necessárias (Não se preocupe com essa pasta);
- O **Logs**, onde é armazenado os logs de erro do coletor. Verifique essa pasta de vez em quando para entender possíveis erros que ocorreram;

- E **Leituras**, que é a pasta que será armazenado os arquivos **CSV** com os dados dos sensores.

Também foi criado um arquivo com o nome **.env**, esse é um arquivo oculto no qual guarda as informações inseridas no terminal, inclusive a senha de aplicativo, portanto ao verificar as informações presentes nele cuidado para não mostrar sua senha. Apenas o usuário que executou o instalador pode ler esse arquivo, portanto, caso queira alterar alguma informação inserida anteriormente, como o email remetente, a senha, o tempo de execução, a quantidade de cada leitura, etc, você pode fazer isso com os comandos:

```
cat .env
```

Para ler as informações presentes no arquivo. E também o comando:

```
nano .env
```

Que abrirá um editor de texto onde é possível alterar a informação que desejar, use o atalho **Ctrl+x,** depois pressione a tecla **"y"** e por fim **Enter** para sair e salvar as alterações.

Pronto, agora a cada hora, minuto 0 (15:00, 16:00, 17:00, etc), o script de coleta será executado em segundo plano e os arquivos gerados podem ser verificados na pasta. Caso algum erro ocorra repetidamente um alerta será enviado por email.

Exemplo de uma coleta presente em um arquivo **CSV**:

```
timestamp,sensor_id 1 sensor_type DS18B20 unit C,sensor_id 2
sensor_type DS18B20 unit C,sensor_id 3 sensor_type DS18B20 unit C
2026-03-03 16:55:04,28.875,28.5625,28.625
2026-03-03 16:55:23,28.875,28.625,28.6875
2026-03-03 16:55:41,28.875,28.625,28.75
2026-03-03 16:55:59,28.9375,28.625,28.75
2026-03-03 16:56:18,28.9375,28.6875,28.75
```

6.3 Executando Dashboard

Nesse ponto, com os arquivos gerados pelo coletor na pasta **Leituras**, vamos executar o Dashboard para análise gráfica dos dados. Ainda com o terminal aberto na pasta do projeto, execute o seguinte comando para ativar o ambiente virtual do **Python** que foi criado:

```
source venv/bin/activate
```

Seu terminal deve parecer com isso agora:

```
(venv) usuario@Ubuntu:~/Documentos/Coletor$
```

O **“(venv)”** significa que o ambiente virtual está ativo.

O próximo passo agora é executar o **Dashboard** em si. Existem duas formas de fazer isso: executando em primeiro plano e executando em segundo plano. Vamos às formas:

1. Executando em primeiro plano

No terminal digite o seguinte comando:

```
streamlit run Dashboard.py
```

Um servidor web será iniciado e o **Dashboard** deve abrir automaticamente no seu browser. O terminal estará ocupado com o servidor rodando, portanto se fechá-lo o servidor vai fechar junto. Para fechar o servidor depois que terminar de usar o **Dashboard** e poder usar o terminal novamente, pressione a combinação de teclas **“Ctrl+c”** no terminal.

2. Executando em segundo plano (Background)

O mesmo comando passado anteriormente será usado aqui, mas dessa vez com o operador **“&”** logo em seguida. Esse operador sinaliza o sistema que o processo do servidor rodará em segundo plano, liberando o terminal. Logo, digite:

```
streamlit run Dashboard.py &
```

No terminal aparecerá a seguinte mensagem:

```
[1] 61480
```

Esse é o PID (*Process Identifier*) do processo que está rodando o servidor, anote-o pois será usado para matar esse processo depois que terminar de usar o **Dashboard**.

Com o servidor rodando é possível fechar o terminal com segurança sem que o mesmo seja finalizado. Portanto, para matar o processo, abra o terminal

novamente e digite o comando: “**kill 61480”**, com isso o processo será finalizado e o servidor fechado.

Depois de executar uma das formas sugeridas deve aparecer a seguinte mensagem no terminal:

```
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.0.1:8501
```

Isso indica que o servidor foi iniciado e pode ser acessado por essas duas URLs. A primeira só pode ser acessada em sua máquina, já a segunda pode ser acessada por qualquer dispositivo dentro da mesma rede da máquina que o servidor está rodando.

Por fim, no **Dashboard** será possível visualizar um gráfico interativo com as leituras dos sensores obtidas nos arquivos **CSV** gerados, filtrar por tipo de sensor e fazer a média das leituras selecionando o **ID** dos sensores. A interface deve se parecer com isso:

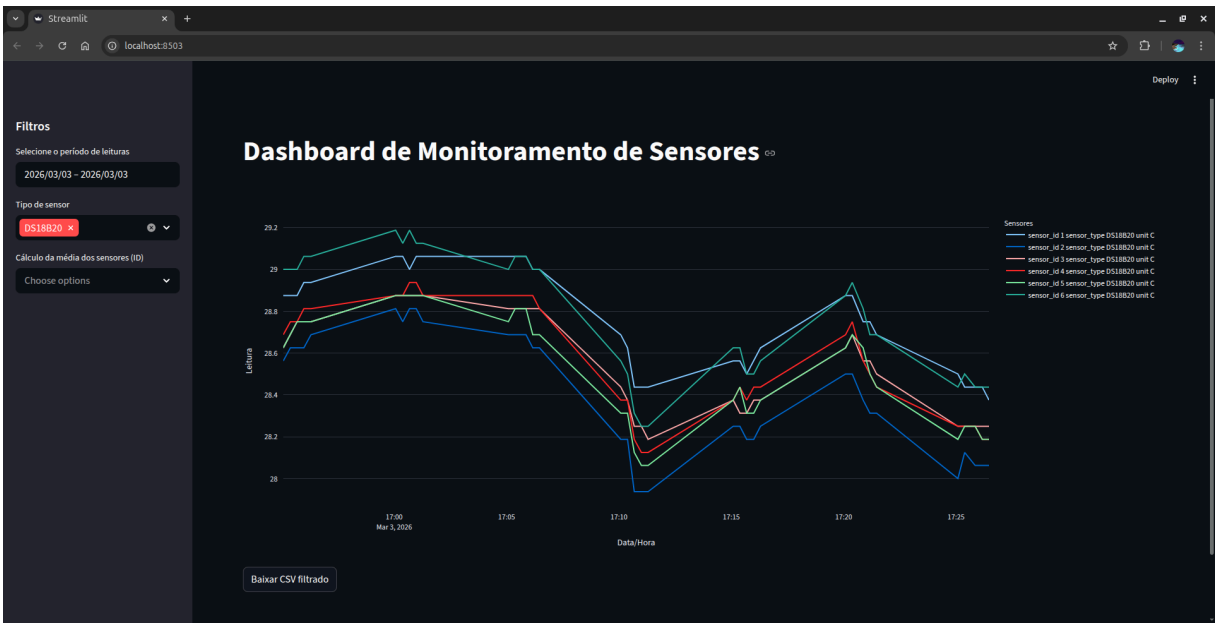


figura 6.6

Para iniciar o **Dashboard** novamente abra o terminal assim como na **figura 6.4** e repita os passos desta subseção.

7. Expansões e Melhorias

- Aderir a suportes ou estruturas, como cases, para proteção e isolamento dos componentes eletrônicos.
- Adicionar mais sensores, visando o aumento de precisão do Sistema.
- Fazer a calibração e testes necessários para garantir a exatidão e precisão dos sensores.

8. Referências

ROBOCORE. **Primeiros passos com o sensor DS18B20 no Arduino**. Robocore.net, [s.d.]. Disponível em: https://www.robocore.net/tutoriais/primeiros-passos-ds18b20-arduino. Acesso em: 29 ago. 2025.

ELETROGATE. **Sensores DHT11 e DHT22**. Blog Eletrogate, [s.d.]. Disponível em: https://blog.eletrogate.com/sensores-dht11-dht22/. Acesso em: 29 ago. 2025.

ARDUINO E CIA. **Sensor de luminosidade BH1750 (LUX) com Arduino**. Arduino e Cia, [s.d.]. Disponível em: https://arduinoecia.com.br/bh1750-sensor-de-luminosidade-lux/. Acesso em: 29 ago. 2025.

BLOG DA ROBÓTICA. **Como utilizar o sensor de umidade do solo com o Arduino**. Blog da Robótica, 6 out. 2022. Disponível em: https://www.blogdarobotica.com/2022/10/06/como-utilizar-o-sensor-de-umidade-do-solo-com-o-arduino/. Acesso em: 29 ago. 2025.

ROBOCORE. **Leitura de umidade do solo com sensor capacitivo**. Robocore.net, [s.d.]. Disponível em: https://www.robocore.net/tutoriais/leitura-umidade-solo. Acesso em: 29 ago. 2025.

MAKERHERO. **ESP8266: Tutorial Arduino com módulo Wi-Fi/Bluetooth**. MakerHero Blog, [s.d.]. Disponível em: https://www.makerhero.com/blog/esp8266-arduino-tutorial/. Acesso em: 29 ago. 2025.

ESPRESSIF SYSTEMS. **ESP8266 AT Instruction Set**. Espressif Documentation, [s.d.]. Disponível em: https://www.espressif.com/sites/default/files/documentation/4a-esp8266_at_instruction_set_en.pdf. Acesso em: 29 ago. 2025.